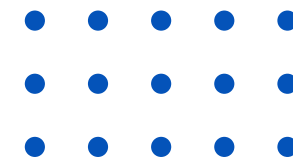




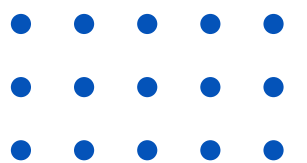
Team AckNet



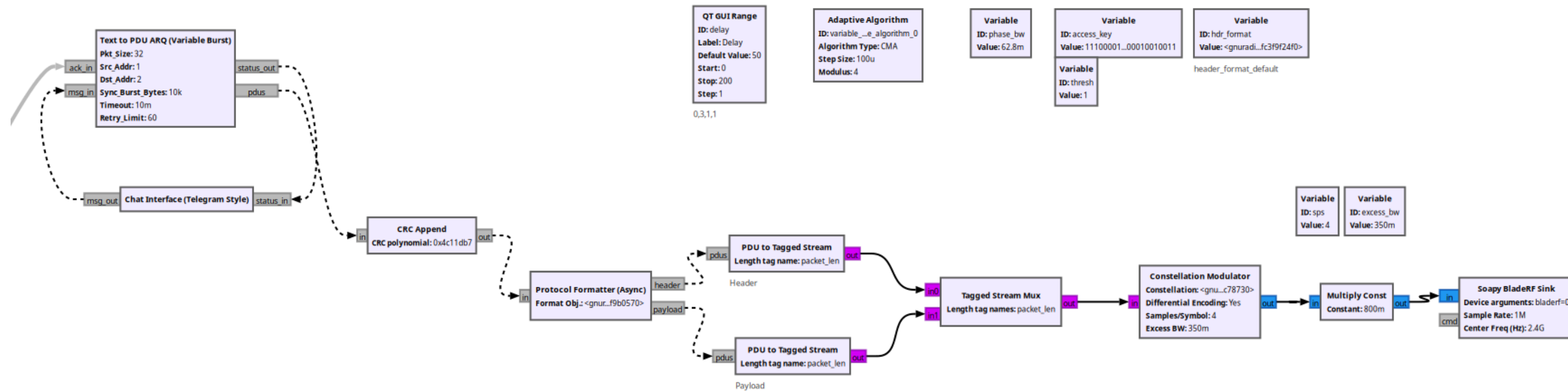
EN2130- Course Project Two-Way Digital Paging System Using Software Defined Radios

- 230476T-Perera G.M.B
- 230012U-Abeywardhana T.C.W
- 230469B- Peiris E.A.S.S
- 230017N-Adhikari A.H.C.S

📍 Department of Electronic & Telecommunication Engineerig
University of Moratuwa, Sri Lanka

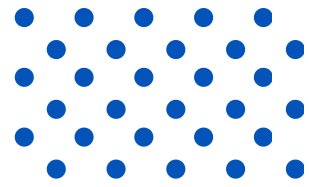


Message Transmitter Block

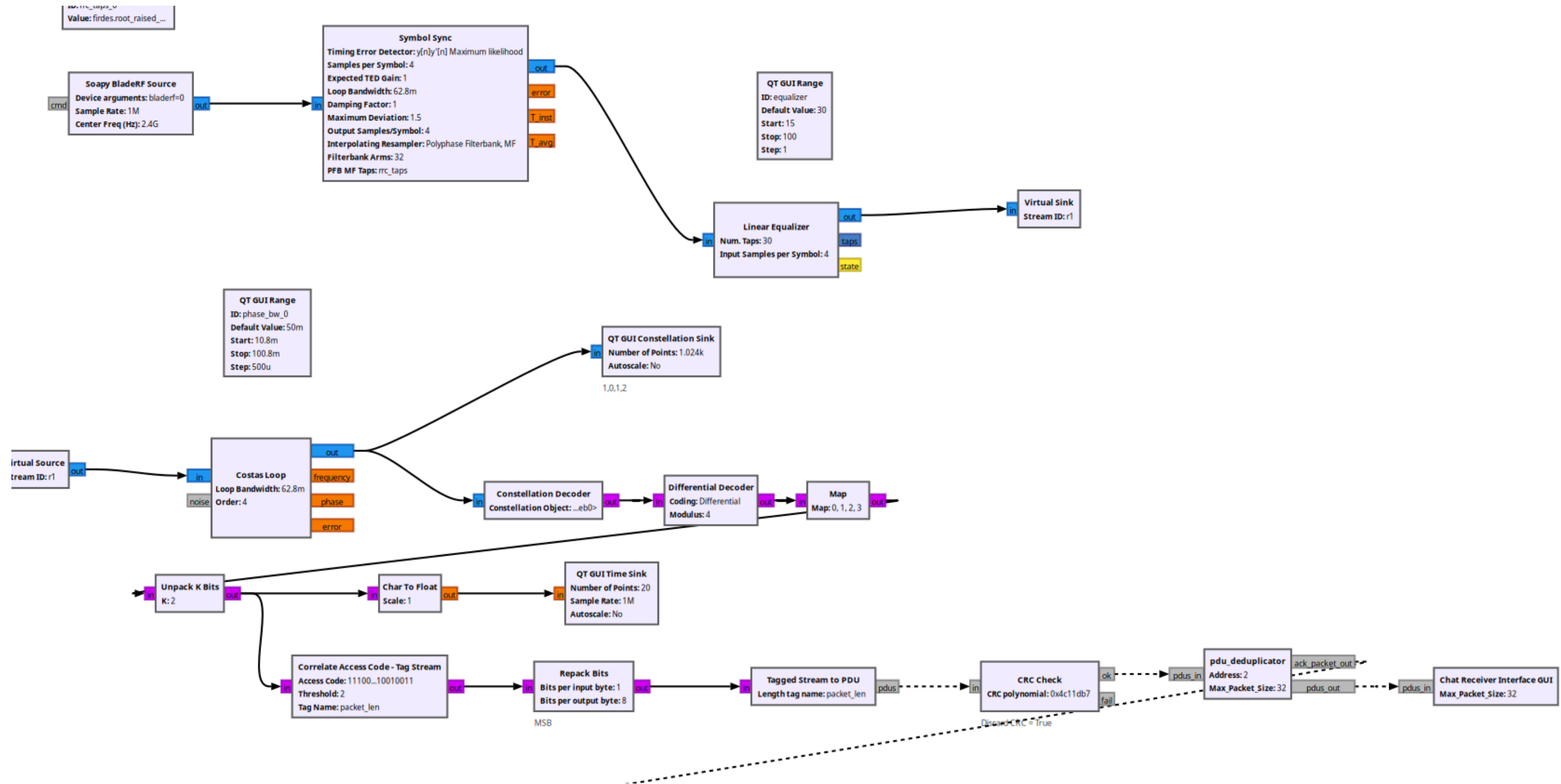


- **Final Packet**

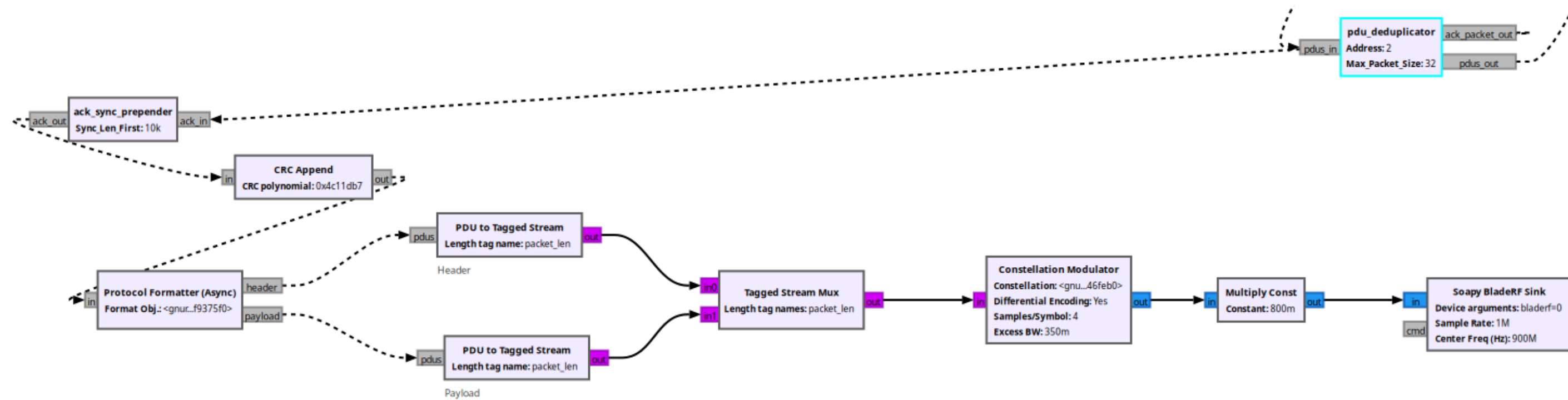
8 Byte Preamble	1 Byte Source Address	1 Byte Destination Address	1 byte Packet Number	29 Byte Payload	4 Byte CRC
--------------------	-----------------------------	----------------------------------	----------------------------	--------------------	---------------



Message Receiving Block



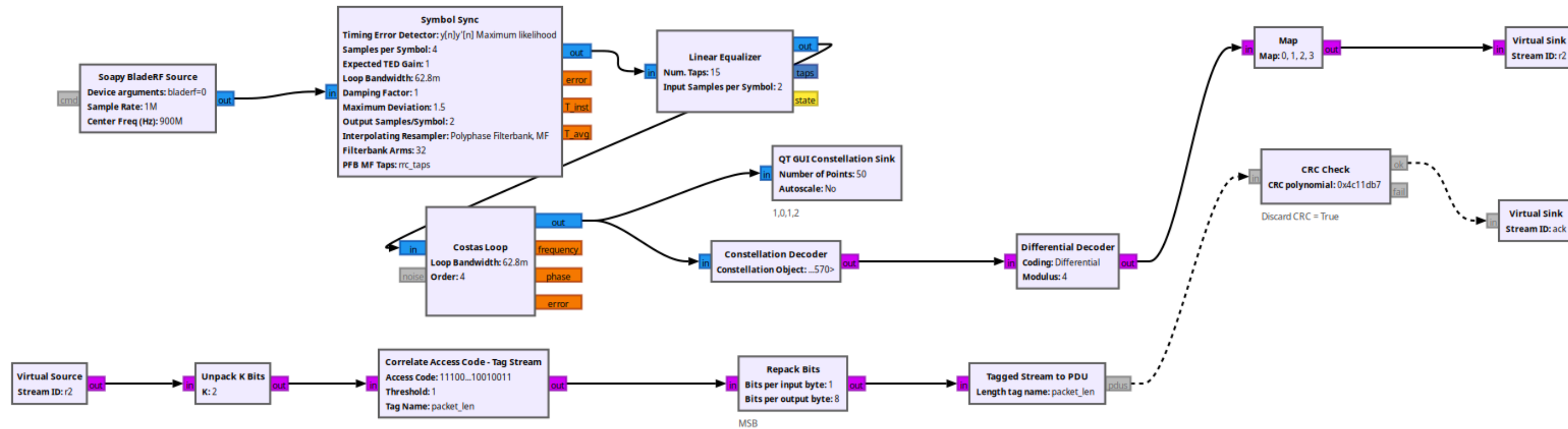
ACK Sending



ACK Packet

0XAA	1 Byte Source Address	1 Byte Destination Address	1 byte Packet Number
------	-----------------------------	----------------------------------	----------------------------

ACK Recieving



Main Challenges and Measurements

1. Sync Issues in Hardware Implementation

Although the simulation achieved perfect synchronization, the hardware implementation failed to sync. The primary reason identified was the use of a very small PDU size. To resolve this, we considered two options:

- Transmitting a continuous byte stream for synchronization.
- Sending a larger PDU to achieve reliable synchronization.

Selected Approach for Synchronization

- Among the two options, we selected Option 2.
- We increased the PDU size by adding synchronization bytes.
- This ensured the receiver had sufficient data length to achieve reliable synchronization.

Main Challenges and Measurements

2. Bandwidth Challenge in Sending Synchronization Bytes

- Another challenge was determining how often to send synchronization bytes.
- Adding sync bytes after every payload packet is bandwidth-inefficient.
- To balance reliability and efficiency, we decided to send synchronization bytes periodically.
- The best trade off we identified was to transmit synchronization bytes around 2 times for a single packet.

3. GUI Implementation Challenge

- another challenge we faced was implementing the graphical user interface (GUI).
- When we developed the GUI externally and connected it to the GNU Radio-generated Python script, sending each message required re-running the entire script.
- This caused significant delays because the system had to reconnect to the BladeRF every time.
- To eliminate this overhead, we integrated the GUI directly inside the GNU Radio project as a custom Python block.

Main Challenges and Measurements

4. Reliable ACK Transmission Challenge

- We also encountered difficulties in sending ACKs reliably.
- The root cause was the same synchronization issue observed on the transmission side.
- To resolve this, we added synchronization bytes periodically to the ACK packets as well.
- This helped maintain alignment and improved ACK reliability.

5. Symbol Synchronization Method Used

- We used Symbol Sync instead of Polyphase Clock Sync.
- Polyphase Sync selects the nearest timing step, which makes it an approximation-based method.
- Symbol Sync mathematically computes the exact symbol timing, providing significantly higher precision.
- This improved the overall synchronization accuracy of our system.

Main Challenges and Measurements

6. Message Termination and Receiver Reset

- Another challenge was indicating to the receiver that a complete message had been fully delivered.
- Without a clear end-of-message signal, the receiver could not determine when to stop expecting additional packets.
- To solve this, we introduced a termination packet at the end of every message.
- When received, this termination packet resets the receiver's next expected packet number, ensuring clean message boundaries and proper synchronization.



Team AckNet

Thank You

For your attention to this presentation.